

الـ JIT مش بيتترجم الملف كله، بيتترجم الفانكشن وقت ما تُستدعى لأول مرة بس الكود المترجم (Native Code) بيتحفظ في الـ RAM، وأي استدعاء ثاني لنفس الفانكشن بيتنفذ فوراً بدون إعادة ترجمة و تحويل الـ IL لـ Native Code مسبقاً وقت الـ Build للتقليل من وقت التشغيل الترجمة بتتم على مستوى الفانكشن (Method-Level) مش الملف كله مرة واحدة.

بتتم مرة واحدة فقط لكل Runtime Session عند أول استدعاء للفانكشن، وطول ما البرنامج شغال بيتم استخدام الكود المترجم الجاهز من الـ RAM.

الـ JIT بيولد تعليمات حسب البتات؛ لو النظام bit-64، ينتج Native Code بيستفيد من (Registers) المساحة الأكبر.

(، وده بياخد مساحة أكبر في الذاكرة لكن بيدي كفاءة أعلى. bit في Bytes-64 إلى bit 8 في Bytes-32) بيكبر (من 4 Pointers حجم)

```
using System;

//using static System.Console;

namespace Csharp1
{
    internal class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello World!");

            Console.ReadKey();

        }
    }
}
```

Compilation with .NET Framework vs .NET Core

وأنا بتعلم .NET ، من الحاجات اللي بدأت أفهمها بشكل مختلف هي إن طريقة بناء وتشغيل الـ Applications اتطورت مع تطور الـ .NET نفسه.

في البداية كان عندنا .NET Framework.

كان بيستخدم لبناء وتشغيل تطبيقات Windows والـ Web Services ، وكان مرتبط بشكل أساسي ببيئة ([2][Microsoft Learn]). Windows.

لكن مع زيادة الحاجة إلى تشغيل التطبيقات على أنظمة مختلفة، وظهور الـ Cloud واحتياجات الـ Modern Applications ، ظهرت الحاجة إلى تصميم جديد.

وهنا ظهر .NET Core.

.NET Core. اتبنى ليكون:

- Cross-Platform
- Open Source
- Modular
- Lightweight

· مناسب للـ Cloud Applications.

وده معناه إن الـ Application ممكن يشتغل على:

Windows

Linux

macOS

بدل ما يكون مرتبط بـ Windows فقط ([3][Microsoft Learn]).

ومن الحاجات اللي لفنت نظري في .NET Core. إنه اتبنى بشكل Modular.

يعني بدل ما يكون عندي Framework كبير وكل مكوناته موجودة مع بعض، أقدر أستخدم الـ Components والـ Libraries اللي محتاجها في الـ Application.

وده بيدي مرونة أكبر في تصميم وتشغيل التطبيقات. كما أن .NET Core. اعتمد على توزيع المكتبات كـ packages ، ومنها ([4][Microsoft Learn]). NuGet packages.

كمان كان التحول إلى Open Source خطوة مهمة.

Microsoft فتحت مصدر .NET Core. وسمحت للمجتمع بالمشاركة في تطويره، وده ساعد في بناء Ecosystem مفتوح يدعم الـ ([5][Microsoft for Developers]). Cross-Platform Development.

طيب إيه علاقة كل ده بالـ Compilation ؟

في الـ .NET Framework. والـ .NET. الحديث، الكود اللي بنكتبه باستخدام C# بيتحول أثناء الـ compilation إلى (IL) Intermediate Language ، وبعدها الـ runtime بيقوم بتنفيذ الـ IL باستخدام الـ JIT compiler.

يعني بشكل مبسط:

C# Code



Compiler



IL



JIT Compiler



Machine Code



Execution

الاختلاف الأساسي مش إن واحد فيهم بيعمل Compilation والثاني لا.

الآتين عندهم. Compilation

الاختلاف الأكبر كان في تصميم الـ platform والـ runtime وطريقة توزيع المكونات ودعم الأنظمة المختلفة.

وده خلاني أفهم إن تطوير .NET مش مجرد تغيير اسم من ` .NET Framework ` إلى ` .NET Core `.

كان فيه تغيير في طريقة تصميم الـ platform نفسها عشان تبقى مناسبة للـ Cross-Platform Development والـ Cloud والـ Modern Applications.

والنهارده المسار اتطور أكثر، وبقي الاسم الأساسي .NET. بداية من .NET 5 ، والـ .NET Core أصبح جزءًا من تاريخ وتطور المنصة الحديثة([1][Microsoft Learn]).

دي كانت واحدة من الحاجات اللي اتعلمتها ضمن **Digital Egypt Pioneers Initiative - DEPI** ، وكل Topic جديد في .NET بيخليني أفهم أكثر إيه اللي بيحصل خلف الكود اللي بكتبه.

لسه مكمل في التعلم والتطبيق، وإن شاء الله أشارك تجارب ومواضيع أكثر من رحلتي في .NET والـ Backend Development.

#DotNet #DotNetCore #CSharp #Programming #BackendDevelopment #DEPI #LearningJourney #SoftwareDevelopment

Report about .NET Versions, Namespace, .NET Core and Solution

1. .NET Versions

.NET is a software development platform created by Microsoft. It provides tools, libraries, and runtimes that developers use to build different types of applications.

The .NET platform has evolved over the years. The main versions can be divided into two stages.

.NET Framework

.NET Framework was the original implementation of .NET. It was released by Microsoft in 2002 and was mainly designed for Windows applications.

The first version was:

.NET Framework 1.0 → 2002 → C# 1.0

The .NET Framework continued to receive updates until:

.NET Framework 4.8 → 2019 → C# 7.3

.NET Framework was widely used for Windows applications and web applications.

However, it had some limitations, especially because it was mainly associated with Windows.

.NET Core

Microsoft introduced .NET Core as a new implementation of .NET.

The first version was:

.NET Core 1.0 → 2016 → C# 6.0

Then the versions continued:

.NET Core 3.1 → 2019 → C# 8.0

.NET 5 → 2020 → C# 9.0

.NET 6 → 2021 → C# 10.0

.NET 7 → 2022 → C# 11.0

.NET 8 → 2023 → C# 12.0

.NET 9 → 2024 → C# 13.0

Starting with .NET 5, Microsoft dropped the word "Core" and continued the platform under the name .NET.

The evolution of .NET also came with improvements to the C# language, including new features, syntax improvements, and what we can call syntax sugar that makes code easier to write and understand.

2. .NET Framework vs .NET Core

One of the main reasons for introducing .NET Core was to address some limitations of the traditional .NET Framework.

.NET Framework

Mainly Windows-based

Developed as a large framework

Commonly used for Windows and web applications

Not designed from the beginning for cross-platform development

.NET Core

Cross-platform

Open source

Modular

Lightweight

Suitable for cloud applications

Supports Windows, Linux, and macOS

Allows developers to use only the components they need

This made .NET Core suitable for modern application development and applications that need to run on different operating systems.

The development later continued with .NET 5 and the newer versions of .NET, creating a unified platform for modern .NET development.

3 .Namespace

A Namespace is a way to organize related classes, interfaces, structures, and other types in a .NET application.

It helps developers organize their code and avoid naming conflicts.

For example:

```
namespace StudentManagement
```

```
}
```

```
    class Student
```

```
}
```

```
    public string Name;
```

```
{
```

```
{
```

Here:StudentManagementis the Namespace.

And:Studentis the class inside that Namespace.

Why do we use Namespace?

Organize the application

Group related classes

Avoid naming conflicts

Make large projects easier to manage

Improve code readability

4. .NET Core

.NET Core is a modern, cross-platform implementation of the .NET platform created by Microsoft.

It was introduced in 2016 with .NET Core 1.0.

One of the main goals of .NET Core was to provide a modern platform that could run on different operating systems.

For example:

Windows

Linux

macOS

Main Features of .NET Core

Cross-Platform

Applications can run on different operating systems.

This is different from the traditional .NET Framework, which was mainly designed for Windows.

Open Source

.NET Core was released as an open-source project.

This allows developers and the community to contribute to the platform and view its source code.

Modular

.NET Core follows a modular approach.

Instead of depending on one large framework containing everything, applications can use the libraries and components they need.

This can help make applications easier to manage and deploy.

High Performance

.NET Core was designed with modern application development and performance in mind.

It became widely used for Web APIs, web applications, cloud applications, and backend systems.

Cloud Ready

.NET Core was designed with modern cloud applications in mind.

This makes it suitable for applications that need to be deployed in environments such as cloud platforms and containers.

5. Solution in .NET

A Solution is a container that organizes one or more projects in a .NET development environment such as Visual Studio.

These concepts work together when developing a .NET application.

A Solution can contain multiple Projects.

Each Project can contain multiple Namespaces.

Each Namespace can contain multiple Classes and other types.
